

AMAZON ML CHALLENGE 2026

Business Entity Resolution: Implementation Plan

Problem, dataset findings, solution design and a build plan to the 27 September deadline

Prepared for	Arya Ambekar and team
Challenge window	25 Sep 2026, 00:00 IST to 27 Sep 2026, 23:59 IST
Metric	Macro-averaged F0.5 per Source 1 entity (precision weighted 2x)
Data analysed	Full training and test files (12.5M train records, 11.7M test records)
Core idea	Assign every Source 2/3 record to at most one owner, then choose each entity's match set to maximise expected F0.5

Figures in this plan come from the team's EDA run on the complete dataset. Score expectations are not predictions of leaderboard rank; they are targets to measure against on a local validation split.

Contents

1	The problem in plain words
2	Rules, deliverables and hard constraints
3	Dataset findings
4	What the findings mean: seven design decisions
5	Solution architecture
6	Stage-by-stage specification
7	Handling France, the unseen country
8	Validation and the local metric
9	Build plan to the deadline
10	Compute, memory and runtime plan
11	Submission strategy
12	Risks and mitigations
13	Code layout and the final submission package
14	Mapping to the official documentation template
A	Appendix: the expected-F0.5 decoder
B	Appendix: compliance checklist

1. The problem in plain words

Amazon receives information about businesses from three independent lists. The same real business shows up in several lists, but written differently each time, and no shared ID connects the copies. Our job is to work out which records describe the same business.

Source 1 is the clean master list: every business appears in it exactly once. For each Source 1 business we must return every Source 2 and Source 3 record that refers to it. The answer may be several records, one, or none.

One real example from the training data

Record	Name	Address
S1 (master)	Springdale, City Of	2007 Robyn Road, Springdale, AR
S2 match	Springdale, City Of Co	007 ROBYN RD, SPRINGDALE, AR
S2 match	Springdale, Cigy Of	SPRINGDALE, 007 ROBYN RD, AR
S3 match	Springdale, Of City	02007 Robyn Road, Springdale, Arkansas
S3 match	*** Springdale City Of	Robyn Rd, Springdale, Arkansas
S3 match	springdalecity.com #39256	02007 Robyn Road, Springdale, Arkansas

All six rows are the same organisation. They contain a typo (Cigy), a character swap (Springdale), junk prefixes (***), a website instead of a name, reordered words, abbreviations (Rd, AR) and a corrupted house number (2007 becomes 007 and 02007). A good system has to see through all of that while still telling apart the 253 different businesses called “Primary Care Group”.

Why it matters in the real world

Duplicate business identities cause split payment histories, broken seller analytics and fraud blind spots. Merging two different businesses is worse: one can receive the other's money, reviews or compliance status. That is why the challenge scores with F0.5, which punishes wrong merges more than missed ones.

2. Rules, deliverables and hard constraints

Item	Requirement
Leaderboard upload	matching_results.tsv : one row per test Source 1 entity, tab-separated, comma-separated ID lists, empty list allowed
Blocking file	candidate_pairs.tsv : the exact candidate set the final model scores. Not scored, but audited for recall ceiling and reduction ratio. Every matched ID must appear in it
Submissions	At most 5 per day. Keep a version history: shortlisting depends on submitted solutions
Leaderboards	Public (subset of test) during the event; final ranking uses the Private leaderboard
Final package	Zip containing output/ (both TSVs), code/business_entity_resolution/ (src/, README.md, pinned requirements.txt) and the filled Documentation_template.md
Model licence	Final model must be MIT or Apache 2.0 licensed and at most 8B parameters
Fair play	No external databases, APIs, geocoders or internet augmentation. Only the provided data
Validator	Run utils/validate_submission.py before every upload; it prints PASS or a list of problems
Open country set	Country is an open set of string labels. Do not hard-code or one-hot {US, India}; France must be handled

The scoring formula, simplified

For one Source 1 entity with k predicted IDs, t of them correct and n true matches: **$F0.5 = 1.25 \cdot t / (k + 0.25 \cdot n)$** . A true singleton scores 1.0 only if we predict nothing; an entity with real matches scores 0 if we predict nothing. Scores are averaged over all Source 1 entities.

What this means in practice, for a typical entity with $n = 4$ true matches:

Prediction	Score	Lesson
All 4 correct	1.000	Perfect
3 correct, nothing wrong	0.938	Missing one match costs about 6 points
2 correct, nothing wrong	0.833	
4 correct + 1 wrong	0.833	One wrong merge costs about 17 points, nearly 3x a miss
3 correct + 1 wrong	0.750	
Nothing	0.000	Silence is fatal when matches exist (94% of entities)

So the metric is not simply “be cautious”. With an average of 3.7 matches per non-singleton entity, recall matters a lot too. The right answer depends on how confident we are about each candidate, which is exactly what the decoder in Section 6.6 computes.

3. Dataset findings

All numbers below come from the team's EDA run over the complete files (runtime about 200 seconds).

3.1 Size and country mix

File	Rows	US	India	France	Size
train S1	2,206,821	1,323,633	883,188	–	210 MB
train S2	5,034,616	3,016,817	2,017,799	–	489 MB
train S3	5,285,603	3,170,056	2,115,547	–	504 MB
train ground truth	2,206,821				127 MB
test S1	1,732,544	663,106	809,986	259,452	175 MB
test S2	4,887,273	1,871,330	2,312,565	703,378	510 MB
test S3	5,082,316	1,945,701	2,405,000	731,615	506 MB

France is 15% of the test Source 1 entities and has no training labels at all. Test also has more Source 2/3 records per Source 1 entity (about 5.5 to 5.8 versus 4.7 in training), so the share of distractors or matches per entity may shift. The pipeline must not rely on training-set base rates without checking them on test.

3.2 Ground-truth structure

Check	Result	Consequence
Singletons (no matches)	5.58% (123,247 entities)	94% of entities have matches; predicting empty is rarely right
Matches per entity	Mean 3.67 over non-singletons, max 11; mode 3	Output sets of several records, not a single best match
Per source	Mostly 1-3 from S2 and 1-3 from S3	Rank S2 and S3 candidates separately as well as jointly
Records linked to more than one S1	0 of 7,638,365 positive pairs	Strict one-to-one: each S2/S3 record has at most one owner
Unmatched S2 / S3 records	26.6% / 25.4%	About a quarter of records are distractors that belong to nobody
Country agreement in true matches	100%	Block strictly within country (as an open string label)
Ground truth coverage	100% of train S1, no missing IDs	Labels are clean and complete

3.3 How similar are true matches?

Signal (true pairs, 200k sample)	Value
Exactly equal name after simple cleaning	25.7%
Name word-overlap (Jaccard) p10 / p25 / p50 / p75	0.00 / 0.50 / 0.67 / 1.00
No shared name word at all	14.5%
Address word-overlap p10 / p25 / p50 / p75	0.25 / 0.43 / 0.67 / 0.82
Both addresses contain numbers	87.2%
Share at least one number	79.8%
Both have numbers but share none	7.4% (corrupted numbers, PMB / PO Box filler)
India median name / address overlap	0.60 / 0.77
US median name / address overlap	0.75 / 0.56

The two countries fail in opposite ways: Indian records keep long, detailed addresses but names drift (local scripts, suffix changes), while US records keep names but addresses lose components. A single similarity threshold cannot serve both, which is why a learned model with separate name and address evidence is needed.

Note on the India name figure: the EDA cleaner removed non-Latin characters, so names written in Devanagari, Telugu, Kannada or Bengali scored zero overlap. The true India name similarity after transliteration will be higher. French accents were folded to plain letters (é to e) rather than dropped.

3.4 Hard negatives: the same name, different businesses

39.3% of Source 1 records share their cleaned name with at least one other Source 1 record. The most common are generic practice names: “primary care group” (253 times), “ear nose throat group” (251), “pediatric group” (222). For these, the name carries almost no information and the address must decide. The model therefore gets a **name-genericness** feature (how many Source 1 entities share this name) so it learns when to trust the name.

3.5 Noise catalogue (observed in real examples)

Noise type	Examples from the data	Handling
Typos, swaps, leetspeak	Cigy, Pdrvate, Sepciialsist, Cardi0logy, 5afe, Springda1e, D0mbivli	Char n-gram similarity; digit-to-letter folding inside words (0→o, 1→l, 5→s)
Name replaced by a website	kimb1eolvias.com, cardiologysafecare.com, bfprivate.com, familybrighthealth.com	Compare against the name with spaces removed; initials feature; strip TLD and #digits
Junk / unrelated name	Iriecto, Evohalovio (address identical to the entity)	Address and cluster evidence must carry the match
Local-script names	Hindi, Telugu names for Latin S1 names; state names in local script	Rule-based transliteration + dictionary mined from training pairs
Legal suffix drift	Private Limited / Pvt / Ltd / LLP / Inc / Co / Corporation / [INC] / (PC)	Strip legal forms into a separate field; compare core names
Added or dropped words	Family Family, Specialists Specialists, Dr, Services, Center, Limited Service	De-duplicate repeated tokens; containment features, not only Jaccard
Word order	Springdale, Of City; India Trust Charitable	Order-insensitive token features
Junk prefixes	***, <<, --	Strip non-alphanumeric leading characters
Abbreviations	Road/Rd, Street/St, Drive/Dr, Parkway/Pkwy, Texas/TX, Telangana/TG, MH, WB	Mined abbreviation table plus state code map
Corrupted numbers	2007 → 007 / 02007; 607 → 60; 405 → 05; KNO 499/499	Fuzzy numeric match: strip leading zeros, suffix match, digit edit distance
Filler values	null, N/A, NULL, PMB 7452, PO Box 2342, #39256	Remove placeholders; exclude PMB / PO Box numbers from conflict checks
Missing / partial address	3.3% of S2/S3 addresses empty; many keep only city and state	Explicit missing-field flags; model learns to lean on the name
Component reordering	“TX, TERRELL, VIRGINIA ST”; “Karnataka, 50, KNO 499/499 ...”	Bag-of-token and number-set comparisons, not positional
City vs county / locality	Floyd County vs Willis; Lancaster County vs Weems; Jalna inserted	Low weight for admin tokens; street + number carry more weight

France test records show the same generator at work: “<< Team Ecole”, accent injection (“Àmicale”, “Àrt”), abbreviations (R. / Rue, AV / Avenue), legal forms (SARL, SAS, SASU, SCI, S.A.), “& Fils / & Frères”, “5 bis”, numbers in brackets “(41) Rue ...”, and region versus department swaps (Hauts-de-France vs Nord). Because the noise process looks shared across countries, features built on noise patterns rather than on country-specific vocabulary should transfer.

4. What the findings mean: seven design decisions

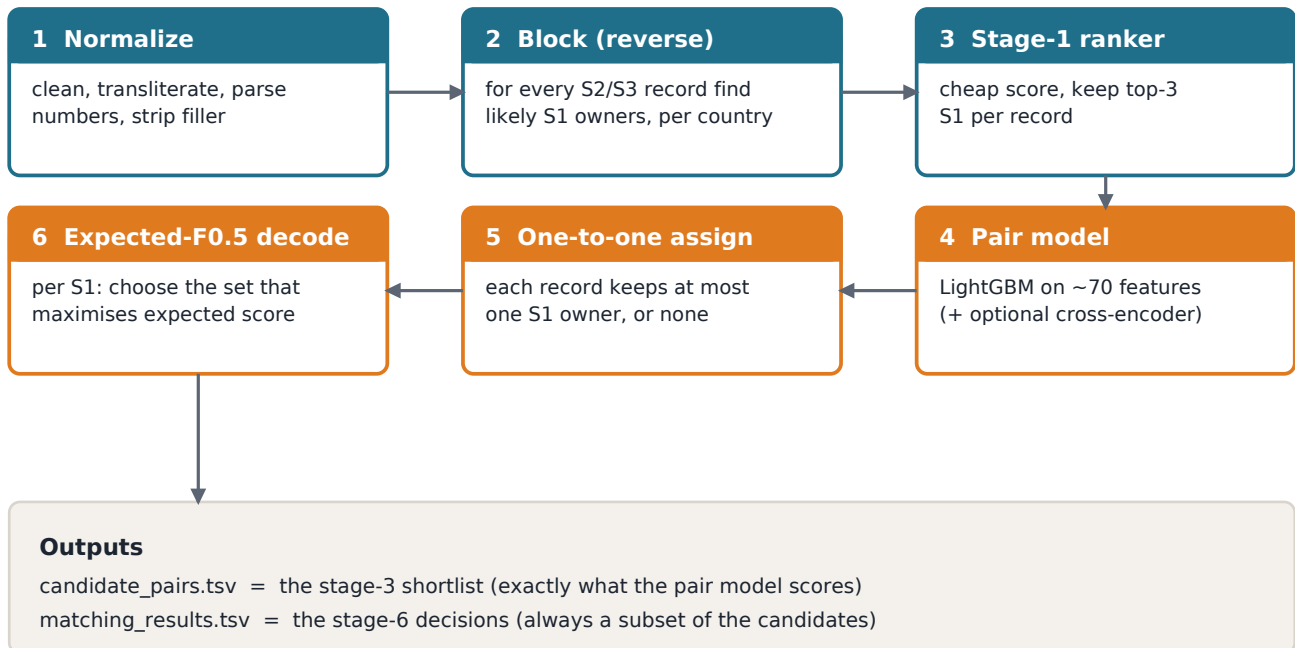
#	Finding	Decision
D1	Zero records linked to two Source 1 entities	Reverse the problem. For each S2/S3 record, decide which single S1 entity owns it, or none. This removes competing claims by construction
D2	F0.5 is computed per entity and averaged	Decode per entity with an expected-F0.5 optimiser instead of one global threshold
D3	100% country agreement	Partition every step by country label, treating the label as an open string
D4	14.5% of true pairs share no name word	Address, numbers, website-name and cluster evidence are first-class, not tie-breakers
D5	39% of S1 names repeat	Genericness features; addresses decide between same-name entities
D6	France has no labels, 15% of test	Country-agnostic features, data-mined dictionaries, leave-one-country-out validation
D7	~12M records per split	Two-stage candidate pipeline: cheap ranking first, full features only on a small shortlist

What makes this approach different

Most teams will build blocking, similarity features, a classifier and a 0.5 threshold. This plan adds three things that follow directly from the data and the metric: (1) record-to-owner assignment that exploits the proven one-to-one structure, (2) metric-optimal set selection per entity, and (3) normalisation dictionaries learned from the data itself, including self-mined French abbreviations and cross-script name mappings, so the system generalises to a country it has never been trained on. Each piece is measurable with an ablation, which gives the methodology document real evidence rather than claims.

5. Solution architecture

The pipeline runs independently per country. Stages 1 to 3 shrink roughly 12 million records into a shortlist; stages 4 to 6 make the decisions.



Stage	Input → output	Why it exists
1 Normalize	raw text → clean fields: core name, legal form, compact name, number set, street tokens, city/state tokens, flags	Removes the noise catalogue before any comparison
2 Block	each S2/S3 record → up to ~15 possible S1 owners (union of several passes)	Sets the recall ceiling; must stay above ~97% pair recall
3 Stage-1 ranker	~15 → top 3 owners per record	Keeps the expensive model affordable; this set is candidate_pairs.tsv
4 Pair model	(record, owner) → match probability	Learns how name, address and context evidence combine
5 Assign	per record: best owner, or none	Enforces the one-to-one rule and removes duplicate claims
6 Decode	per S1: probabilities → final set	Chooses the set with the highest expected F0.5

6. Stage-by-stage specification

6.1 Normalisation

Every record is converted into a fixed set of derived fields, computed once and cached as Parquet.

Field	How it is built
name_clean	Unicode NFKC, transliterate non-Latin scripts to Latin, fold accents, lowercase, strip leading junk (**, <-, --), remove bracketed tags ([INC], (PC)), collapse repeated words
legal_form	Extracted legal suffix mapped to a canonical code (pvt ltd → PRIVATE_LIMITED, llc, inc, sarl, sas, sci ...)
name_core	name_clean without legal forms and filler words; leetspeak folded inside alphabetic words

Field	How it is built
name_compact	name_core with spaces removed (matches website-style names such as cardiologysafecare)
is_domain, initials	Flag for .com/.in/.fr style names; initials of name_core (bf for Bangalore Farmer)
addr_tokens	Address tokens after placeholder removal (null, N/A), abbreviation expansion, state-code mapping
numbers	Number tokens with leading zeros stripped; PMB / PO Box / # numbers moved to a separate filler set
num_keys	Blocking keys: last 3 digits of each house number, zero-padded (2007, 007 and 02007 all give 007)
city_state	Best-guess locality and state tokens (low weight, since they vary legitimately)
flags	addr_empty, name_nonlatin, source (S2 / S3), token counts

Dictionaries learned from data, not typed by hand

- **Abbreviation pairs.** For true training pairs, align tokens that differ between the two addresses or names (Rd ↔ Road, Pvt ↔ Private, TG ↔ Telangana). Keep pairs that recur often. This covers US and India.
- **Cross-script names.** Training contains parallel pairs such as a Latin S1 name matched to a Hindi- or Telugu-script S2 name. After rule-based transliteration, align the tokens to learn corrections for common words (the local-script spellings of Private and Limited, state names). No outside data is used.
- **French abbreviations (self-mined).** Run the pipeline once on France, take only very confident matches, align differing tokens (R ↔ Rue, AV ↔ Avenue, BD ↔ Boulevard), add them to the dictionary and run again. This uses only the unlabelled test inputs, which is allowed, and should be documented openly.
- **Small seed list.** A short, documented list of legal forms per language (SARL, SAS, SASU, SCI, EURL, S.A., & Fils / & Frères) is domain knowledge, not external data lookup.

Library choice: prefer permissively licensed transliteration tools (for example *anyascii*, *ISC*, or *indic-transliteration*, MIT) over GPL ones, so the licence review stays clean.

6.2 Blocking (candidate generation)

Blocking runs in the **reverse** direction: each S2/S3 record looks for its possible owner among the S1 records of the same country. The union of the passes below forms the raw candidate set.

Pass	Key or method	Catches
A Address number	num_key + first street token (exact join)	Most US and India pairs; survives name junk and websites
B Rare name tokens	Rarest 1-2 core-name tokens + city token; skip blocks larger than a cap	Pairs with damaged or missing addresses
C Char TF-IDF search	Char 3-4-grams of name_compact + numbers + city; sparse top-k search per country chunk	Typos, word order, spacing, domains
D Compact / initials	name_compact prefix and initials against S1 compact names	Website-style names (bfprivate.com)
E Address-only	Char TF-IDF on addresses for records whose name is junk or non-Latin	Iriecto / Evohalovio cases

Stage-1 ranker. A small LightGBM model on about 10 cheap features (TF-IDF cosines, number overlap, pass flags) ranks each record's raw candidates, and the top 3 per record are kept. Regrouped by S1 entity, this shortlist is written as `candidate_pairs.tsv`.

Targets to measure on validation: raw-union pair recall of at least 98%, shortlist recall of at least 97%, and a reduction ratio above 99.999% relative to all same-country pairs. Report these numbers in the documentation; the organisers analyse exactly this.

6.3 Pair features (about 70)

Group	Features
Name	Token Jaccard and containment on <code>name_core</code> ; Jaro-Winkler; token-sort and partial ratios; char 3-gram cosine; IDF-weighted overlap; compact-name similarity; initials match; legal-form equal / compatible / conflicting; <code>is_domain</code> and non-Latin flags on either side
Address	Token Jaccard and containment; IDF-weighted overlap; char cosine; street-token match; city match; state match (after code mapping); token-count difference; empty-address flags
Numbers	Exact number overlap; fuzzy overlap (suffix / leading-zero / one-digit edit); conflict count excluding filler; largest matching number length
Genericness	How many S1 entities share this core name; IDF of the rarest shared token; name length
Context (from the one-to-one structure)	Rank of this S1 among the record's candidates; score gap to the record's best alternative owner; rank of this record among the S1's candidates; number of candidates on each side
Cluster (second pass)	Maximum similarity between this record and the S1's other high-confidence records (lets a junk-named record join via an address twin)
Source	S2 or S3 (their formats differ: S2 uses upper-case addresses, S3 spells out state names)

Country is deliberately **not** a feature, so the model cannot learn country-specific shortcuts that fail on France.

6.4 Pair model

- **Main model:** LightGBM binary classifier (MIT licence) trained on shortlist pairs, grouped cross-validation by S1 entity. About 300k to 500k sampled S1 entities with all their candidate pairs is enough and keeps training under an hour.
- **Calibration:** isotonic regression on out-of-fold predictions, because the decoder needs true probabilities, not just a ranking.
- **Optional semantic signal (day 3, only if time allows):** fine-tune a small multilingual cross-encoder (for example multilingual-e5-small, MIT, or xlm-roberta-base, MIT) on name + address pairs and feed its score to LightGBM, scoring only the uncertain band (probability 0.2–0.8) to limit GPU time. Keep it only if the ablation shows a gain.

6.5 One-to-one assignment

For each S2/S3 record, keep only its highest-probability owner. Its match probability becomes $p(\text{record} \rightarrow \text{owner})$ and the record is dropped from every other entity. Because the training data never violates one-to-one, this step can only remove false positives. A record whose best probability is low simply stays unassigned, which is how the 26% of distractors are handled.

6.6 Expected-F0.5 decoding

For each S1 entity, collect the probabilities $p_1 \geq p_2 \geq \dots$ of the records assigned to it. The best set is always the top-k by probability for some k (including k = 0). For each k, the decoder computes the expected F0.5 exactly using the Poisson-binomial distribution of how many chosen and unchosen records are true, plus a small prior for true matches that blocking missed. It then picks the k with the highest expected score.

This replaces the usual global threshold with a decision tuned to each entity's evidence. It also handles singletons naturally: the empty set wins only when all candidates are weak. Details are in Appendix A. The prior for missed matches and a probability temperature are the only two tuned knobs, both set on validation.

7. Handling France, the unseen country

France makes up 259,452 of the 1,732,544 test entities (15%), and poor handling there would cap the score. The plan attacks it on four fronts.

Front	Action
Features that transfer	No country feature; similarity, number and genericness features behave the same in any Latin-script language
Normalisation	Accent folding (removes injected accents such as Àmicale), French legal forms, self-mined street abbreviations, handling of bis / ter numbers and bracketed numbers
Honest validation	Leave-one-country-out: train on US only and score India, then the reverse. The drop measures how much a new country costs, and each fix is judged by whether it shrinks that drop
Sanity checks on test	Compare France's distribution of predicted matches per entity, probability histogram and empty-set rate with US and India. Large gaps signal a normalisation gap to fix before the final submission

Why leave-one-country-out is a strong story for judges

It is the only way to measure generalisation to an unseen country with the data available. Reporting “in-country F0.5 vs held-out-country F0.5, before and after each fix” shows the team designed for the distribution shift the organisers deliberately built into the test set.

8. Validation and the local metric

- **Split:** hold out 15% of training S1 entities (stratified by country and by number of matches). Blocking still searches the full training S1 index, so validation entities compete with training entities exactly as test entities will.
- **No leakage:** the pair model never sees positive labels of held-out entities. Stage-1 and stage-2 models are trained only on training-fold entities.
- **Metric:** implement the official macro F0.5 exactly (per entity, singletons included, empty-vs-empty = 1.0) and check it against the worked example in the problem statement (0.714).
- **Report at every stage:** blocking recall, shortlist recall, pair AUC and log-loss, F0.5 with a 0.5 threshold, and F0.5 with the decoder. The last two side by side are the proof of the decoder's value.
- **Trust order:** local validation first, public leaderboard second. The private leaderboard decides the final rank.

```
def f05_entity(pred, true):
    pred, true = set(pred), set(true)
    if not true:
        # singleton
        return 1.0 if not pred else 0.0
    if not pred:
        return 0.0
    t = len(pred & true)
    return 1.25 * t / (len(pred) + 0.25 * len(true))

def macro_f05(pred_map, true_map): # dicts: sl_id -> list of ids
    return sum(f05_entity(pred_map.get(k, []), v) for k, v in true_map.items()) / len(true_map)
```

9. Build plan to the deadline

The deadline is **Sunday 27 September, 23:59 IST**. The plan front-loads a valid submission, then adds the components in order of expected value. Every block ends with something measurable.

Day 1 · Friday 25 September (evening and night)

Block	Task	Done when
1	Set up the environment (Kaggle notebook with the data as a private dataset, or a local machine with 16 GB+ RAM). Convert all TSVs to Parquet	All six files load in seconds
2	Normalisation v1: cleaning, legal forms, numbers, num_keys, compact name	Fields cached for train and test
3	Validation split + local metric (checked against 0.714)	Metric verified
4	Blocking passes A + C, reverse direction, per country; measure recall on validation	Pair recall reported
5	Baseline: rule score (name + address cosine) → one-to-one → fixed threshold. Write both TSVs, run the validator, submit	Submission 1 scored

Day 2 · Saturday 26 September

Block	Task	Done when
1	Add blocking passes B, D, E; stage-1 ranker; freeze the shortlist size by the recall target	Shortlist recall $\geq 97\%$
2	Full feature set (name, address, numbers, genericness, context)	Feature table built for sampled training entities
3	LightGBM + isotonic calibration; grouped cross-validation	Validation AUC and log-loss logged
4	One-to-one assignment + expected-F0.5 decoder; compare with the best global threshold	Submission 2 (model) and 3 (model + decoder)
5	Transliteration + mined dictionaries (abbreviations, cross-script)	India F0.5 gain measured
6	France: accent folding, legal forms, leave-one-country-out runs, French self-mining	Submission 4 ; France sanity checks pass

Day 3 · Sunday 27 September

Block	Task	Done when
1	Cluster (second-pass) features; retrain	Ablation logged
2	Optional cross-encoder on the uncertain band, only if GPU time allows and the ablation shows a gain	Kept or dropped by evidence
3	Tune the decoder's two knobs on validation; error analysis of top false positives and false negatives	Submission 5-7
4	Freeze the pipeline by ~18:00. Regenerate outputs end to end from a clean run of the README steps	Reproducible run matches the submitted file
5	Fill the documentation template (Section 14), package the zip, final validator run, final upload with a buffer before 23:59	Zip and final submission done

Suggested split of work (adjust to team size)

Workstream	Owns	Hands over
A · Data & blocking	Parquet conversion, normalisation, dictionaries, blocking, stage-1 ranker, candidate_pairs.tsv	Shortlist tables + recall numbers
B · Modelling	Features, LightGBM, calibration, cluster features, optional cross-encoder	Calibrated probabilities per pair
C · Decisions & delivery	Metric, validation, assignment, decoder, France checks, submissions log, README, documentation	Final TSVs, zip, write-up

10. Compute, memory and runtime plan

Concern	Plan
Environment	Kaggle notebook (about 30 GB RAM, free GPU) with the competition data uploaded as a private dataset, or a local machine with at least 16 GB RAM. Colab's standard RAM may be tight for full-size joins
Memory	Parquet + categorical dtypes; process one country at a time; float32 features; release frames between stages
Blocking cost	Sparse top-k search (e.g. sparse_dot_topn) over char n-grams, chunked by country and by blocks of query rows; cap block sizes in key-based passes
Feature cost	RapidFuzz vectorised pairwise functions (multi-threaded) on the shortlist only (about 3 owners × ~10M records on test, roughly 30M pairs)
Training	Sample 300-500k training entities; LightGBM on CPU is minutes to under an hour
Checkpointing	Save every stage's output to disk so a crash never costs more than one stage

Runtime targets are estimates; time the first full run of each stage on day 1 and cut shortlist size or feature count if the full test run would exceed about two hours.

11. Submission strategy

- Each upload tests **one** change, so the leaderboard delta can be attributed. Never spend an upload on an untested format.
- Keep a log: submission number, date and time, git commit, change, local F0.5 (overall and per country), public score.
- Use the public score as a sanity check, not an optimisation target: it covers only part of the test set, and chasing it risks overfitting that the private leaderboard will expose.

- The final choice is the version with the best local validation score whose public score is consistent with it.
- Leave at least one daily slot unused until late in the day in case a bug needs a corrected upload.

12. Risks and mitigations

Risk	Impact	Mitigation
Blocking misses true matches	Hard ceiling on recall	Five complementary passes; recall measured per pass and per country
France performs far below US/India	Up to 15% of entities affected	Section 7; leave-one-country-out; test-set sanity checks
Probability shift on test (more records per entity)	Decoder over- or under-predicts	Check predicted-set-size distributions on test; tune temperature conservatively
Memory or time overrun at full scale	Missed submissions	Per-country processing; timed dry runs on day 1; cached stages
Overfitting the public leaderboard	Private rank drops	Decisions driven by local validation
Invalid output format	Wasted upload	Validator before every upload; empty rows kept for singletons
Licence or fair-play issue	Disqualification	Only MIT/Apache models; no external lookups; transliteration and dictionaries documented as rule-based and data-mined
Pipeline not reproducible	Fails the package review	One entry point; pinned requirements; clean end-to-end rerun before freezing

13. Code layout and the final submission package

```

<team_name>_submission.zip
├── output/
│   ├── matching_results.tsv
│   └── candidate_pairs.tsv
├── code/business_entity_resolution/
│   ├── src/
│   │   ├── config.py           # paths, seeds, shortlist size, decoder knobs
│   │   ├── io_utils.py         # TSV <-> Parquet, writers for both output files
│   │   ├── normalize.py        # cleaning, transliteration, numbers, legal forms
│   │   ├── dictionaries.py     # abbreviation / cross-script / French mining
│   │   ├── blocking.py         # passes A-E + stage-1 ranker
│   │   ├── features.py         # pair, context and cluster features
│   │   ├── train.py            # LightGBM + isotonic calibration
│   │   ├── assign_decode.py    # one-to-one assignment + expected-F0.5 decoder
│   │   ├── metric.py           # official macro F0.5
│   │   └── run_pipeline.py     # single entry point: data -> both TSVs
│   ├── README.md              # exact commands, hardware used, runtimes
│   └── requirements.txt        # pinned versions
└── Documentation_template.md  # filled in

```

Every function gets a docstring, as the guidelines require. The README lists the exact commands, the hardware used and the runtime of each stage, so a reviewer can reproduce both output files from the raw data.

14. Mapping to the official documentation template

Template section	What to put there (from this plan)
1 Executive summary	Reverse one-to-one assignment + expected-F0.5 decoding + data-mined normalisation; headline validation score
2.1 Problem analysis	Section 3 findings: singleton rate, zero multi-owner records, 14.5% no shared name word, 39% repeated names, noise catalogue
2.2 Solution strategy	Approach type: hybrid (blocking + GBDT + structured decoding). Core innovation: D1, D2, D6
3 Candidate generation	Passes A-E, stage-1 ranker, pair counts, recall per pass and per country, reduction ratio
4 Matching model	Feature table from 6.3; LightGBM + isotonic; threshold method = expected-F0.5 decoder instead of a threshold
5 Results & error analysis	Ablation table (baseline → model → + one-to-one → + decoder → + dictionaries → + cluster); per-country scores; leave-one-country-out; common false positives and negatives
6 Conclusion	What worked, what did not, lessons on metric-aware decoding
Appendix A	Code structure from Section 13 and the entry point
Appendix B	Recall curves, calibration plot, predicted-set-size histograms per country

Appendix A · The expected-F0.5 decoder

For an entity with candidates sorted by probability $p_1 \geq \dots \geq p_m$, choosing the top k gives $F0.5 = 1.25 \cdot T / (k + 0.25 \cdot (T + U + B))$, where T is the number of true matches among the chosen k , U the number among the unchosen, and B the number blocking never found. Assuming independence, T and U follow Poisson-binomial distributions computed by a simple dynamic program, and B uses a small prior estimated from validation recall.

```
import numpy as np

def poibin(ps):
    """Distribution of the number of successes among independent Bernoullis."""
    dist = np.zeros(len(ps) + 1); dist[0] = 1.0
    for p in ps:
        dist[1:] = dist[1:] * (1 - p) + dist[:-1] * p
        dist[0] *= (1 - p)
    return dist

def best_set(ps, miss_prior=(0.97, 0.03)):
    """ps sorted descending. miss_prior: P(B=0), P(B=1). Returns best k."""
    ps = np.asarray(ps)
    all_zero = np.prod(1 - ps) * miss_prior[0] # expected score of empty set
    best_k, best_val = 0, all_zero
    for k in range(1, len(ps) + 1):
        T, U = poibin(ps[:k]), poibin(ps[k:])
        val = 0.0
        for b, pb in enumerate(miss_prior):
            for t in range(1, k + 1):
                for u in range(len(U)):
                    n = t + u + b
                    val += pb * T[t] * U[u] * 1.25 * t / (k + 0.25 * n)
        if val > best_val:
            best_k, best_val = k, val
    return best_k, best_val
```

Candidate lists per entity are short (typically under 10 after assignment), so this exact computation is cheap even for 1.7M entities; it can be vectorised or run in parallel per country if needed. Independence is an approximation; the temperature knob on probabilities absorbs part of the error and is tuned on validation.

Appendix B · Compliance checklist

Check	Status to confirm before final upload
Every test S1 entity has exactly one row in both files	Validator PASS
Only S2/S3 IDs that exist in test; no duplicates within a list	Validator PASS
$\text{matching_results} \subseteq \text{candidate_pairs}$ for every entity	Validator shows no warning
candidate_pairs is the exact set the final model scored	Written by the same code path that feeds the model
Models MIT / Apache 2.0 and $\leq 8\text{B}$ parameters	LightGBM (MIT); any cross-encoder licence recorded in the README
No external data, APIs or geocoding	Dictionaries are mined from provided data; seed legal-form list documented
Country treated as an open set	No hard-coded {US, India}; France present in outputs
Reproducible from README + requirements	Clean end-to-end rerun completed
Documentation template filled	All sections, including ablations and error analysis
Submission history kept	Log with versions, commits and scores